

Authentication

Sam Ogden

Updated 2026-04-28 14:38

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Lecture Objectives

- After this lecture, you should be able to:
- Explain three main authentication methods
- Consider the challenges of each method

Some definitions before we begin

- **Principal:** who or what is asking for permission
- **Object:** the resource being asked for
- **Credential:** tracking of permissions granted

How to assign credentials?

- We need to figure out who to let run
- And what permissions they should have
- Authentication only matters if the OS can tie identity to access



Main ways to check

Authentication usually comes from one of three kinds of evidence

- 1. What you know**
- 2. What you have**
- 3. What you are**

What do you know?

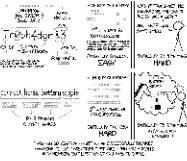
Passwords

What do you know?

Before we trust passwords, we need to ask:

- What makes a good secret?
- How do we store secrets?
- How do we check secrets?
- How do we protect secrets?

Password entropy



How do we check them?

Input		Digest
Any	SHA256 with length = 256	0000 0000 0000 0000 0000 0000 0000 0000
Transaction data e.g. 1000000000	SHA256 with length = 256	0000 0000 0000 0000 0000 0000 0000 0000
Block data e.g. 1000000000	SHA256 with length = 256	0000 0000 0000 0000 0000 0000 0000 0000
Block data e.g. 1000000000	SHA256 with length = 256	0000 0000 0000 0000 0000 0000 0000 0000
Block data e.g. 1000000000	SHA256 with length = 256	0000 0000 0000 0000 0000 0000 0000 0000

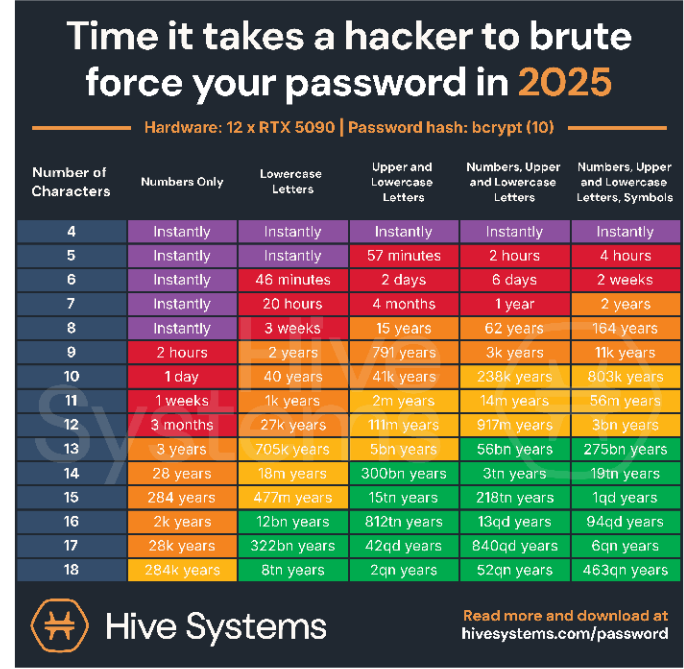
How do we protect secrets?

Say somebody manages to get our password file....

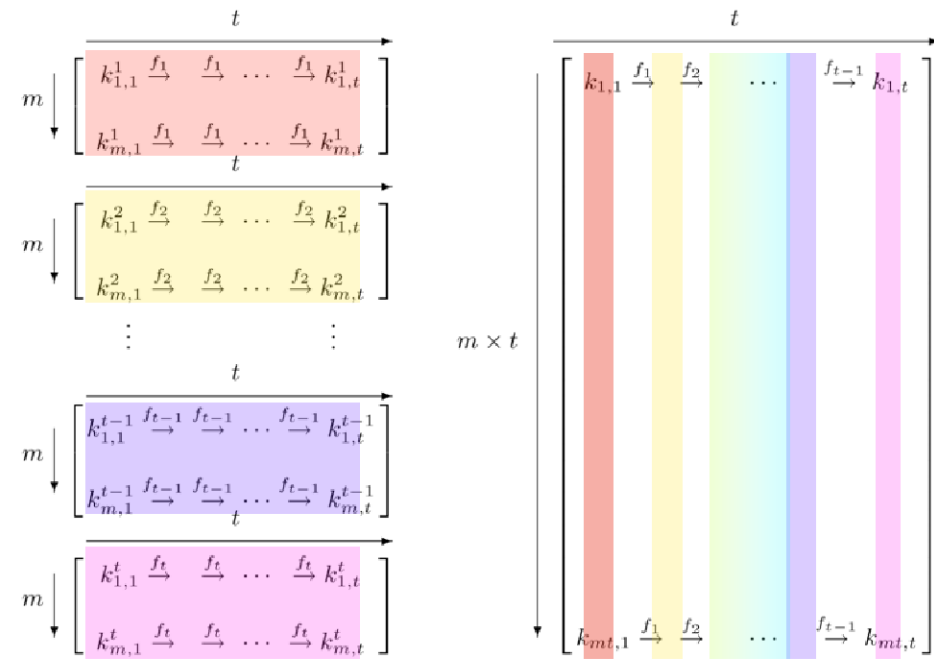
If it's hashed, do we have a problem?

YES! They can try all the hashes!

So how do we fix this?



Time to crack a password of a given length



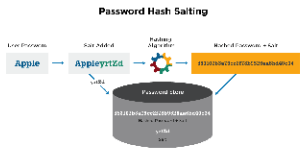
Rainbow Table illustration presented at Crypto 2003

We add a salt



- A **salt** is a little extra information added to every password
 - Effectively makes it longer
- Each user has their own **salt**
 - It doesn't need to be private
- They effectively remove the benefits of precomputation (aka rainbow tables)

How does *salting* work?



What do you know? Key Takeaways

- Use an expertly designed hashing function
- Protect your password file
- Always add salt to the hash
- Passwords that people can remember are better than ones they write down

What do you have?

Cards and Keys

What you have: What is it?

- Authentication based on having a specific thing
 - A concert ticket
 - A lanyard
 - Your physical ATM card
 - Your cell phone
 - A security dongle

What problems are there?

- What happens if you lose your device?
- What happens if somebody steals your device?
- How do you prove you're still you?
- Can somebody else use it?
- How often does it update?
- Will people use it?

What you are Biometrics

What are biometrics?

- Measurements of what makes you you
- Retinal scan (Mission Impossible)
- Palm print (Whole Foods)
- Thumb print (iPhone SE)

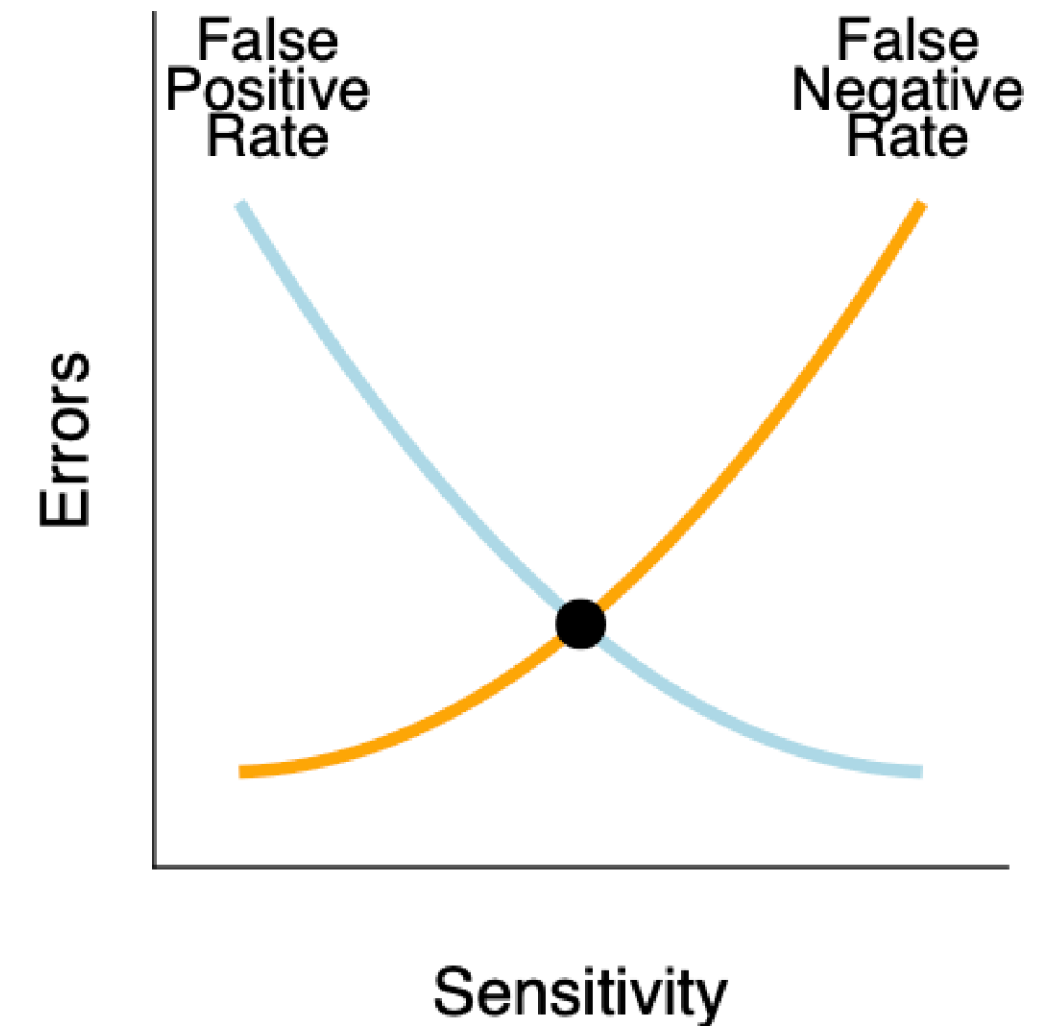
Biometrics Challenges

Are these the same cat?



Sensitivity

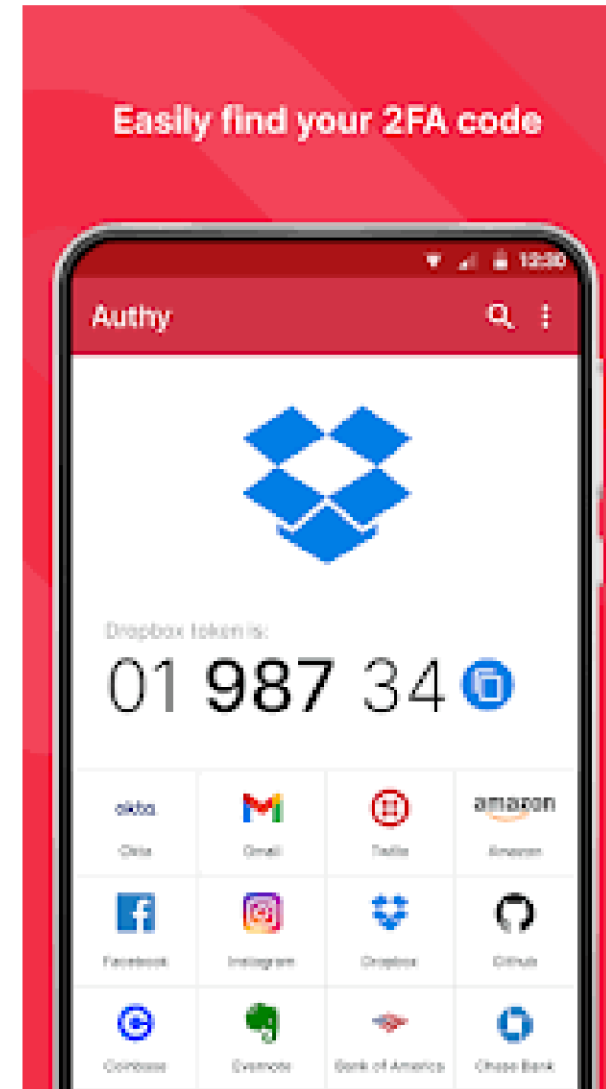
- We need to strike a balance between being too relaxed and too stringent
- Context matters!
 - How mission-critical is this?
 - What is our mission?



How can we make it better?

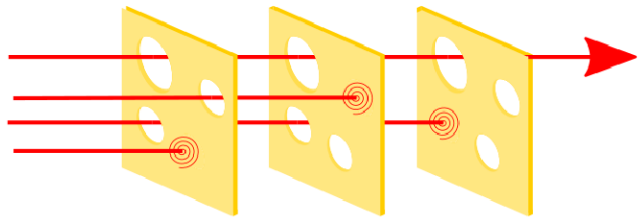
Authentication is stronger when we combine things that fail in different ways.

- Password + phone
- Multi-Factor Authentication



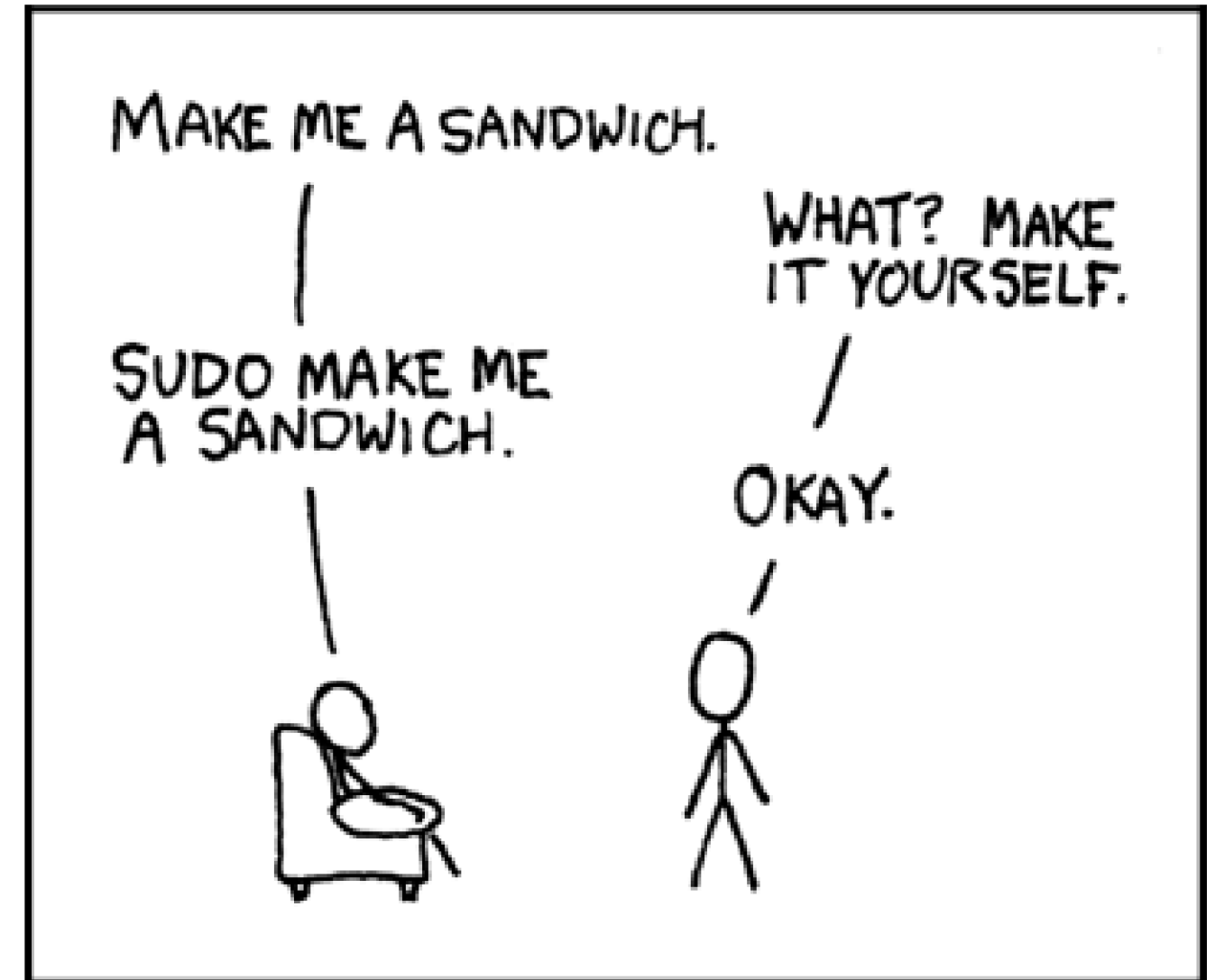
How to design a system

- The swiss cheese model says add lots of layers to help protect yourself
- If something makes it through one layer, stop it before the next



How to authenticate the system?

- Sometimes you want to run tasks not as yourself
- And you don't want to sign in
- How do we do this?



Summary

- Three main authentication methods:
 - what you know
 - what you have
 - what you are
- These work best when combined
- Need to balance strictness and usability